

Uniwersytet Wrocławski
Wydział Fizyki i Astronomii

Oliwer Urbaniak

System zarządzania pracą drukarki 3D w laboratorium
studenckim.

Praca inżynierska na kierunku
Informatyka Stosowana i Systemy Pomiarowe

Opiekun
dr Bartosz Brzostowski

Wrocław, 18 maja 2026

Spis treści

1	Wprowadzenie	8
1.1	Motywacja i cel pracy	8
1.2	Zakres pracy	8
1.3	Struktura pracy	9
2	Analiza wymagań i przegląd rozwiązań	10
2.1	Wymagania funkcjonalne	10
2.2	Wymagania нефункционалне	10
2.3	Przegląd istniejących rozwiązań	12
2.3.1	Octoprint	12
2.3.2	Mainsail i Fluidt	13
2.3.3	Rozwiązania komercyjne	13
2.3.4	Wnioski	13
3	Wybór technologii	14
3.1	Języki programowania	14
3.1.1	TypeScript / Node.js	14
3.1.2	Python	14
3.2	Framework webowy - Express.js	14
3.3	Baza danych - Azure Cosmos DB	14
3.4	Przechowywanie plików - Azure Blob Storage	15
3.5	Kolejkowanie - Azure Service Bus	15
3.6	Sterowanie urządzeniem - Azure IoT Hub	15
3.7	Frontend - React i Vite	16
3.8	Kontenery - Docker i Docker Compose	16
3.9	CAD - SolidWorks	17
3.10	Porównanie wybranych alternatyw	17
4	Architektura systemu	18
4.1	Ogólna struktura	18
4.2	Diagram architektury	18

4.3	Model danych	18
4.3.1	Użytkownik (kontener users)	18
4.3.2	Zadanie (kontener jobs)	18
4.3.3	Refresh Token (kontener refresh-token)	18
4.4	Cykl życia zadania do wydruku	18
4.5	Bezpieczeństwo i uwierzytelnianie	18
5	Implementacja serwisów backendowych	20
5.1	Auth Service	20
5.1.1	Opis ogólny	20
5.1.2	Endpointy API	20
5.1.3	Kluczowe fragmenty implementacji	20
5.1.4	Weryfikacja e-mail	20
5.2	User Service	20
5.2.1	Opis ogólny	20
5.2.2	Endpointy API	20
5.2.3	Mechanizm autoryzacji	20
5.3	Files Service	20
5.3.1	Opis ogólny	20
5.3.2	Przepływ przesyłania pliku	20
5.4	Queue Service	20
5.4.1	Opis ogólny	20
5.4.2	Listener kolejki	20
5.4.3	Endpointy HTTP API	20
5.5	Printer Service	20
5.5.1	Opis ogólny	20
5.5.2	Harmonogram	20
5.5.3	Monitor gotowości	20
5.5.4	Listener telemetrii	20
5.5.5	Enpointy HTTP API	20
5.6	Video Service	20

6	Implementacja frontendu	21
6.1	Struktura aplikacji	21
6.2	Klient API	21
6.3	Routing i ochrona tras	21
6.4	Ekrany aplikacji	21
6.4.1	Strona główna	21
6.4.2	Dashboard użytkownika	21
6.4.3	Upload i planowanie	21
6.4.4	Kontrola druku	21
6.4.5	Panel administracyjny	21
6.5	Internacjonalizacja	21
6.6	Obsługa motywu	21
7	Agent Raspberry Pi - AddiPi Agent	22
7.1	Opis ogólny	22
7.2	Architektura agenta	22
7.3	Połączenie z IoT Hub	22
7.4	Obsługiwane metody bezpośrednie	22
7.5	Przebieg obsługi metody startPrint	22
7.6	Telemetria	22
7.7	Komunikacja z OctoPrint	22
8	Obudowa urządzenia - projekt CAD	23
8.1	Cel i zakres projektu mechanicznego	23
8.2	Moduł CASE - obudowa Raspberry Pi z kamerą	23
8.2.1	Podstawa projektowa	23
8.2.2	Zakres modyfikacji	23
8.2.3	Mocowanie kamery	23
8.2.4	Struktura plików CASE	23
8.3	Moduł STAND - podstawka regulowana	23
8.4	Złożenie całości	23
8.5	Parametry wydruku	23
8.6	Instrukcja montażu	23

9	Konfiguracja Raspberry Pi	24
9.1	Opis środowiska	24
9.2	Instalacja agenta	24
9.3	Automatyczne uruchamianie agenta przez systemd	24
9.4	Tunel Cloudflare - ekspozycja kamery	24
9.4.1	Instalacja cloudflared	24
9.4.2	Skrypt publikowania URL kamery	24
9.4.3	Cykliczne odnawianie URL	24
9.5	Konfiguracja crontab	24
9.5.1	Uzasadnienie parametrów	24
9.6	Problem z dostępnością sieci przy starcie	24
9.7	Podsumowanie konfiguracji Raspberry Pi	24
10	Wdrożenie i infrastruktura	25
10.1	Infrastruktura chmurowa	25
10.2	Inicjalizacja infrastruktury	25
10.3	Konteneryzacja - Dockerfiles	25
10.4	Docker Compose	25
10.5	Zmienne środowiskowe	25
10.6	Wdrożenie frontendu	25
10.7	Wdrożenie agenta	25
11	Testowanie systemu	26
11.1	Metodologia testowania	26
11.2	Testy jednostkowe - Auth Service	26
11.3	Testy integracyjne przez Postman	26
11.4	Test end-to-end - pełny przepływ druku	26
11.5	Testy wydajnościowe i obserwacje	26
12	Podsumowanie	27
12.1	Osiągnięcia	27
12.2	Napotkane trudności	27
12.3	Możliwości rozwoju	27
12.4	Wnioski	27

Załączniki	30
A Konfiguracja środowiska deweloperskiego	30
A.1 Wymagania	30
A.2 Pierwsze uruchomienie	30
B Struktura repozytoriów	30
C Endpointy API - podsumowanie	30

Streszczenie

Niniejsza praca inżynierska opisuje projekt, implementację oraz wykonanie systemu *AddiPi* - rozproszonego systemu zarządzania pracą drukarki 3D przeznaczonego dla laboratorium studenckiego. System umożliwia użytkownikom przesyłanie plików G-code, planowanie oraz monitorowanie zadań do wydruku, a administratorom - pełną kontrolę nad kolejką i użytkownikami.

Architektura systemu oparta jest na podejściu mikroserwisowym. Wyodrębniono sześć niezależnych serwisów backendowych: uwierzytelniania (Auth Service), zarządzania użytkownikami (User Service), obsługi plików (Files Service), kolejkowania zadań (Queue Service), sterowania drukarką (Printer Service) oraz agenta działającego na urządzeniu Raspberry Pi (AddiPi Agent). Całość uzupełnia aplikacja frontendowa zbudowana w React oraz serwis wideo do podglądu kamery.

Komunikacja między serwisami odbywa się za pośrednictwem protokołu HTTP/REST oraz chmurowej kolejki Azure Service Bus. Dane przechowywane są w Azure Cosmos DB, a pliki G-code w Azure Blob Storage. Sterowanie drukarką realizowane jest przez Azure IoT Hub przy użyciu bezpośrednich wywołań metod (Direct Methods) na Raspberry Pi, na którym działa oprogramowane OctoPrint.

System zrealizowano przy użyciu języków TypeScript (Node.js) i Python. Wdrożenie odbywa się w kontenerach Docker, koordynowanych plikiem Docker Compose. Interfejs użytkownika obsługuje uwierzytelnianie tokenem JWT, przesyłanie plików, planowanie zadań, podgląd kamery w czasie rzeczywistym oraz panel administracyjny.

Słowa kluczowe: mikroserwisy, drukarka 3D, OctoPrint, Raspberry Pi, Azure Iot Hub, Azure Cosmos DB, Docker, TypeScript, Python, REST API, kolejkowanie zadań

1 Wprowadzenie

1.1 Motywacja i cel pracy

Drukarki 3D stanowią obecnie standardowe wyposażenie laboratoriów akademickich, warsztatów studenckich oraz zakładów produkcyjnych zorientowanych na wytwarzanie przyrostowe. Urządzenia te, pomimo rosnącej dostępności i malejących cen, pozostają zasobem współdzielonym - ich liczba jest zwykle ograniczona, a czas drukowania pojedynczego modelu może wynosić od kilkunastu minut do wielu godzin. W środowisku akademickim oraz przemysłowym rodzi to naturalne problemy organizacyjne: konieczność ręcznego umawiania się na czas druku, brak przejrzystości stanu kolejki oraz niemożność zdalnego monitorowania postępu pracy.

Celem niniejszej pracy jest zaprojektowanie i implementacja systemu *AddiPi*, który automatyzuje proces zarządzania drukarką 3D w laboratorium studenckim. System pozwala użytkownikom na:

- przesyłanie plików G-code przez przeglądarkę internetową,
- planowanie druków na określony termin lub dodawanie ich do kolejki,
- śledzenie postępu druku w czasie rzeczywistym, wraz z podglądem kamery,
- otrzymywanie powiadomień o zakończeniu lub błędzie druku.

Administratorzy laboratorium zyskują natomiast narzędzia do zarządzania użytkownikami, przeglądania historii wszystkich zadań oraz sterowania kolejką i urządzeniami.

1.2 Zakres pracy

Praca obejmuje:

1. analizę wymagań systemu i wybór technologii,
2. projekt architektury mikroserwisowej,
3. implementację wszystkich komponentów backendowych i frontendowych,
4. konfigurację infrastruktury chmurowej (Microsoft Azure),
5. wdrożenie systemu przy użyciu kontenerów Docker,

6. testy poprawności działania.

1.3 Struktura pracy

Rozdział 2 zawiera analizę wymagań i przegląd istniejących rozwiązań. Rozdział 3 omawia wybrane technologie i uzasadnia decyzje projektowe. Rozdział 4 opisuje architekturę systemu. Rozdziały 5–7 szczegółowo omawiają implementację poszczególnych komponentów programowych. Rozdział 8 opisuje projekt mechaniczny obudowy w SolidWorks. Rozdział 9 omawia konfigurację Raspberry Pi, automatyzację przez cron i tunel Cloudflare. Rozdział 10 dotyczy wdrożenia i infrastruktury chmurowej Azure. Rozdział 11 przedstawia wyniki testów. Praca kończy się podsumowaniem w rozdziale 12.

2 Analiza wymagań i przegląd rozwiązań

2.1 Wymagania funkcjonalne

Na podstawie analizy potrzeb laboratorium studenckiego sformułowano następujące wymagania funkcjonalne:

Dla użytkowników:

- rejestracja z weryfikacją adresu e-mail (ograniczoną do domeny uczelni @uwr . edu . pl),
- logowanie z użyciem tokenów JWT (access token + refresh token),
- przesyłanie plików G-code (maksymalnie 50 MB, format . gcode),
- planowanie druku na wybrany termin lub dodanie do kolejki ogólnej,
- przeglądanie własnych zadań z filtrowaniem według statusu,
- podgląd postępu druku w czasie rzeczywistym (pasek postępu, temperatura dyszy i podłoża, obraz z kamery),
- anulowanie i ponowne uruchamianie własnych zadań.

Dla administratorów:

- pogląd i zarządzanie wszystkimi użytkownikami systemu,
- zmiana roli użytkownika (użytkownik/administrator),
- podgląd i zarządzanie wszystkimi zadaniami druku,
- potwierdzenie gotowości drukarki do kolejnego zlecenia,
- dostęp do metryk systemowych (liczba zadań według statusu).

2.2 Wymagania нефunkcjonalne

- **Skalowalność** - architektura powinna umożliwiać dodanie kolejnych drukarek bez przebudowy systemu.
- **Niezawodność** - utrata wiadomości z kolejki nie powinna powodować trwałej utraty zadania.

- **Bezpieczeństwo** - dostęp do API chroniony tokenami JWT, hasła przechowywane w formie szyfru bcrypt.
- **Responsywność interfejsu** - aplikacja frontendowa powinna działać zarówno na urządzeniach desktopowych, jak i mobilnych.
- **Czas odpowiedzi** - operacje API powinny odpowiadać w czasie poniżej 2 sekund w warunkach normalnego obciążenia.
- **Przenośność** - wszystkie komponenty opakowane w kontenery Docker, uruchamiane jednym poleceniem.

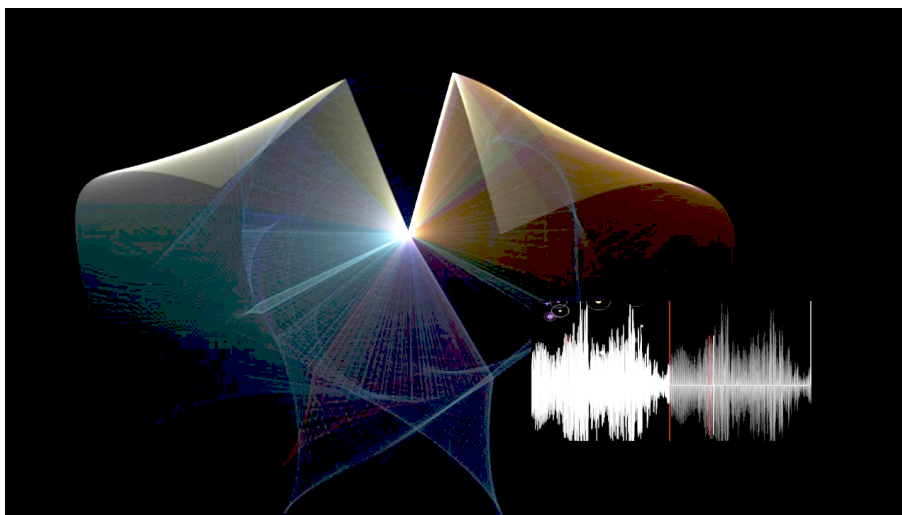
W swojej pracy możesz odnosić się do równań np. do równania (1) lub do rysunków, zarówno wektorowych, patrz rys. (1) jak i bitmapowych (2). Możesz też z powodzeniem cytować literaturę pojedynczo [5] lub kilka pozycji na raz [?, 1]. Jeśli chcesz to robić wygodniej załóż tzw, plik bibtex, ale możesz też pracować tak, jak w tym dokumencie.

$$\rho \left(\frac{\partial \vec{v}}{\partial t} + (\vec{v} \cdot \nabla) \vec{v} \right) = \rho \vec{f} - \nabla p + \mu \Delta \vec{v}, \quad (1)$$

Lorem ipsum dolor sit amet, consectetur adipiscing elit, sed do eiusmod tempor incididunt ut labore et dolore magna aliqua. Ut enim ad minim veniam, quis nostrud exercitation ullamco laboris nisi ut aliquip ex ea commodo consequat. Duis aute irure dolor in reprehenderit in voluptate velit esse cillum dolore eu fugiat nulla pariatur. Excepteur sint occaecat cupidatat non proident, sunt in culpa qui officia deserunt mollit anim id est laborum.



Rysunek 1: Podpis rysunku, który jest obrazem wektorowym (EPS).



Rysunek 2: To jest rysunek drugi, kilkadziesiąt wahadeł podwójnych z synteza dźwięku.

Lorem ipsum dolor sit amet, consectetur adipiscing elit, sed do eiusmod tempor incididunt ut labore et dolore magna aliqua. Ut enim ad minim veniam, quis nostrud exercitation ullamco laboris nisi ut aliquip ex ea commodo consequat. Duis aute irure dolor in reprehenderit in voluptate velit esse cillum dolore eu fugiat nulla pariatur. Excepteur sint occaecat cupidatat non proident, sunt in culpa qui officia deserunt mollit anim id est laborum. .. in the figure 2.2.

2.3 Przegląd istniejących rozwiązań

2.3.1 Octoprint

OctoPrint [4] jest najpopularniejszym oprogramowaniem do zarządzania drukarkami 3D. Działa jako serwer HTTP na Raspberry Pi i udostępnia interfejs webowy, API REST oraz system wtyczek. Umożliwia zdalne sterowanie drukarką, podgląd z kamery i monitorowanie postępu.

OctoPrint jest jednak przeznaczony dla jednego użytkownika zarządzającego jedną drukarką. Nie oferuje mechanizmu kolejkowania zadań wielu użytkowników, systemu uwierzytelniania opartego na rolach ani integracji z chmurą. W systemie AddiPi OctoPrint pełni rolę lokalnego sterownika drukarki, nad którym nadbudowano opisywaną w tej pracy warstwę zarządzania.

2.3.2 Mainsail i Fluiddd

Mainsail i Fluiddd to nowoczesne interfejsy webowe dla firmware Klipper. Podobnie jak OctoPrint, skierowane są do jednego użytkownika i nie przewidują obsługi kolejki wielu użytkowników.

2.3.3 Rozwiązania komercyjne

Oblio, Polar Cloud i Ultimaker Digital Factory to komercyjne platformy zarządzania flotą drukarek 3D w środowisku przemysłowym i edukacyjnym. Oferują zaawansowane funkcje, jednak wiążą się z kosztami subskrypcji i nie pozwalają na pełną kontrolę nad infrastrukturą - co w warunkach laboratorium akademickiego jest istotną przeszkodą.

2.3.4 Wnioski

Żadne z dostępnych rozwiązań o otwartym kodzie źródłowym nie łączy w sobie: systemu uwierzytelniania obsługującego wielu użytkowników, kolejkowania zadań z planowaniem czasowym, integracji z chmurą obliczeniową i sterowaniem przez IoT. System AddiPi wypełnia tę lukę, pozostając jednocześnie w pełni samodzielny i wdrażany lokalnie.

3 Wybór technologii

3.1 Języki programowania

3.1.1 TypeScript / Node.js

Większość serwisów backendowych oraz aplikacja frontendowa zostały napisane w TypeScript [11] - nadzbiorze języka JavaScript dodającym statyczne typowanie. TypeScript znacznie ułatwia utrzymanie dużych baz kodu dzięki wykrywaniu błędów na etapie kompilacji, podpowiedziom IDE i samodokumentowaniu przez typy.

Node.js [13] wybrano ze względu na asynchroniczny, nieblokujący model wejścia-wyjścia, który sprawdza się w serwisach obsługujących wiele równoległych połączeń HTTP oraz komunikację z zewnętrznymi usługami (baza danych, kolejka, IoT Hub).

3.1.2 Python

Agent działający na platformie Raspberry Pi został zaimplementowany w języku Python [15]. Biblioteki `azure-iot-device` oraz `azure-storage-blob` są w tym środowisku powszechnie dostępne i szczegółowo udokumentowane. Python stanowi standardowe rozwiązanie w przypadku aplikacji uruchamianych na systemach wbudowanych oraz mikrokomputerach jednoukładowych (ang. *Single-Board Computer*, SBC).

3.2 Framework webowy - Express.js

Wszystkie serwisy platformy Node.js wykorzystują framework Express.js [12] w wersji 5.0. Rozwiązanie to charakteryzuje się minimalistyczną architekturą i niskim narzutem wydajnościowym. Framework nie narzuca sztywnej struktury katalogów (ang. *unopinionated framework*), co pozwala na elastyczne dostosowanie architektury każdego z mikroservisów do jego specyficznych wymagań. Express.js zapewnia również ustandaryzowane mechanizmy obsługi żądań pośredniczących (ang. *middleware*), routingu oraz zarządzania błędami.

3.3 Baza danych - Azure Cosmos DB

Azure Cosmos DB [8] to w pełni zarządzana baza NoSQL firmy Microsoft, oferująca globalną dystrybucję danych, elastyczne partycjonowanie oraz obsługę wielu modeli API

(SQL, MongoDB, Cassandra, Gremlin, Table). W projekcie AddiPi wykorzystano API SQL (Core), umożliwiające wykonywanie zapytań zbliżonych składnią do języka SQL na dokumentach zapisanych w formacie JSON.

Wybór Azure Cosmos DB został podyktowany następującymi czynnikami:

- brak konieczności zarządzania infrastrukturą serwera bazy danych dzięki wykorzystaniu modelu PaaS (Platform as a Service),
- elastyczny schemat dokumentów umożliwiający przechowywanie danych o różnej strukturze w ramach jednej bazy danych (addipi), z podziałem na odrębne kontenery (users, jobs, refresh-tokens),
- natywna integracja z ekosystemem Microsoft Azure,
- automatyczne indeksowanie wszystkich pól dokumentów.

3.4 Przechowywanie plików - Azure Blob Storage

Pliki G-code przesyłane przez użytkowników są przechowywane w usłudze Azure Blob Storage [7], w kontenerze gcode. Usługa ta zapewnia praktycznie nieograniczoną przestrzeń magazynową, relatywnie niskie koszty przechowywania danych oraz prosty interfejs umożliwiający przesyłanie i pobieranie obiektów binarnych.

3.5 Kolejkiwanie - Azure Service Bus

Azure Service Bus [10] to chmurowa usługa kolejkiwania komunikatów klasy enterprise firmy Microsoft. W systemie AddiPi pełni rolę brokera komunikatów pomiędzy usługami Files Service (producent komunikatów) oraz Queue Service (konsument).

Kolejka print-queue zapewnia semantykę dostarczania wiadomości typu at-least-once. Oznacza to, że w przypadku awarii usługi konsumenckiej komunikat pozostaje w kolejce i może zostać przetworzony powtórnie po ponownym uruchomieniu konsumenta.

3.6 Sterowanie urządzeniem - Azure IoT Hub

Azure IoT Hub [9] to usługa firmy Microsoft przeznaczona do zarządzania urządzeniami Internetu Rzeczy (IoT) w środowisku chmurowym. Umożliwia dwukierunkową komuni-

kację z urządzeniami (Device-to-Cloud oraz Cloud-to-Device), a także wywołanie metod bezpośrednich (Direct Methods), stanowiących synchroniczne wywołania RPC (zdalne wywołanie procedury) wykonane na urządzeniach komunikujących się za pośrednictwem protokołów MQTT lub AMQP.

W projekcie AddiPi usługa Printer Service wywołuje na agencie działającym na urządzeniu Raspberry Pi metody `startPrint`, `cancelPrint` oraz `getStatus`. Agent przesyła następnie telemetry w postaci zdarzeń, takich jak `print_started`, `print_progress` czy `print_completed`, które są odbierane przez usługę Printer Service za pośrednictwem endpointu kompatybilnego z Azure Event Hubs.

3.7 Frontend - React i Vite

Interfejs użytkownika został zbudowany z wykorzystaniem biblioteki React [6] 18 oraz języka TypeScript. Jako serwer deweloperski i narzędzie do budowania aplikacji wykorzystano Vite [17].

Do zarządzania stanem globalnym aplikacji zastosowano bibliotekę Zustand [14], umożliwiającą współdzielenie danych pomiędzy komponentami aplikacji w uproszczony sposób, bez konieczności implementowania rozbudowanej logiki zarządzania stanem.

Warstwa stylów została zrealizowana przy użyciu frameworka Tailwind CSS [16], który opiera się na wykorzystaniu niewielkich klas CSS reprezentujących pojedyncze właściwości stylów interfejsu użytkownika.

3.8 Kontenery - Docker i Docker Compose

Każdy serwis aplikacji posiada własny plik `Dockerfile` realizujący wieloetapowy proces budowania obrazu kontenera (*multi-stage build*). Rozwiązanie to pozwala na ograniczenie rozmiaru finalnych obrazów oraz oddzielenie środowiska kompilacji od uruchomieniowego.

Docker Compose [3] wykorzystano do zarządzania uruchamianiem wszystkich serwisów zarówno w środowisku lokalnym, jak i produkcyjnym. Narzędzie definiuje wspólną sieć `addipi-network` oraz konfigurację zmiennych środowiskowych odczytywanych z pliku `.env`.

3.9 CAD - SolidWorks

Projekt mechaniczny obudowy urządzenia wykonano w programie **SolidWorks** [2], stanowiącym środowisko do parametrycznego modelowania bryłowego oraz projektowania złożeń mechanicznych. Oprogramowanie umożliwia tworzenie asocjatywnych modeli 3D, w których modyfikacja parametrów części powoduje automatyczną aktualizację złożeń oraz dokumentacji technicznej.

Wybór programu SolidWorks był podyktowany dostępnością oprogramowania w ramach licencji studenckiej oraz szerokim wsparciem dla formatów eksportu modeli, takich jak .STEP (neutralny format wymiany modeli CAD), .STL (format wykorzystywany w procesie przygotowania modeli o druku 3D) oraz .3MF (format wymiany zawierający dodatkowe metadane procesu druku).

Projekt został podzielony na pliki .SLDPRT, reprezentujące pojedyncze części, oraz .SLDASM, definiujące kompletne złożenia mechaniczne.

3.10 Porównanie wybranych alternatyw

Tabela 1: Porównanie baz danych rozważanych w projekcie

Cecha	Cosmos DB	PostgreSQL	MongoDB Atlas
Model danych	Dokument (JSON)	Relacyjny	Dokument (BSON)
Zarządzanie	PaaS (bez serwera)	Samodzielny/PaaS	PaaS
Integracja z Azure	Natywna	Ograniczona	Pośrednia
Elastyczny schemat danych	Tak	Nie	Tak
Dostępność bezpłatnego planu	Tak (400 RU/s)	Tak (self-hosted)	Tak (512 MB)

Tabela 2: Porównanie rozwiązań kolejkowania wiadomości

Cecha	Azure Service Bus	RabbitMQ	Apache Kafka
Model wdrożenia	PaaS	Self-hosted	Self-hosted / PaaS
Semantyka dostarczania	At-least-once	At-least-once	At-least-once
Złożoność konfiguracji	Niska	Średnia	Wysoka
Integracja z Azure	Natywna	Ograniczona	Pośrednia
Skalowalność	Automatyczna	Ręczna	Wysoka

4 Architektura systemu

4.1 Ogólna struktura

4.2 Diagram architektury

4.3 Model danych

4.3.1 Użytkownik (kontener `users`)

4.3.2 Zadanie (kontener `jobs`)

4.3.3 Refresh Token (kontener `refresh-token`)

4.4 Cykl życia zadania do wydruku

4.5 Bezpieczeństwo i uwierzytelnianie

5 Implementacja serwisów backendowych

5.1 Auth Service

5.1.1 Opis ogólny

5.1.2 Endpointy API

5.1.3 Kluczowe fragmenty implementacji

5.1.4 Weryfikacja e-mail

5.2 User Service

5.2.1 Opis ogólny

5.2.2 Endpointy API

5.2.3 Mechanizm autoryzacji

5.3 Files Service

5.3.1 Opis ogólny

5.3.2 Przepływ przesyłania pliku

5.4 Queue Service

5.4.1 Opis ogólny

5.4.2 Listener kolejki

5.4.3 Endpointy HTTP API

5.5 Printer Service

5.5.1 Opis ogólny

5.5.2 Harmonogram

5.5.3 Monitor gotowości

5.5.4 Listener telemetrii

5.5.5 Enpointy HTTP API

5.6 Video Service

6 Implementacja frontendu

6.1 Struktura aplikacji

6.2 Klient API

6.3 Routing i ochrona tras

6.4 Ekran aplikacji

6.4.1 Strona główna

6.4.2 Dashboard użytkownika

6.4.3 Upload i planowanie

6.4.4 Kontrola druku

6.4.5 Panel administracyjny

6.5 Internacjonalizacja

6.6 Obsługa motywu

7 Agent Raspberry Pi - AddiPi Agent

7.1 Opis ogólny

7.2 Architektura agenta

7.3 Połączenie z IoT Hub

7.4 Obsługiwane metody bezpośrednie

7.5 Przebieg obsługi metody startPrint

7.6 Telemetria

7.7 Komunikacja z OctoPrint

8 Obudowa urządzenia - projekt CAD

8.1 Cel i zakres projektu mechanicznego

8.2 Moduł CASE - obudowa Raspberry Pi z kamerą

8.2.1 Podstawa projektowa

8.2.2 Zakres modyfikacji

8.2.3 Mocowanie kamery

8.2.4 Struktura plików CASE

8.3 Moduł STAND - podstawka regulowana

8.4 Złożenie całości

8.5 Parametry wydruku

8.6 Instrukcja montażu

9 Konfiguracja Raspberry Pi

9.1 Opis środowiska

9.2 Instalacja agenta

9.3 Automatyczne uruchamianie agenta przez systemd

9.4 Tunel Cloudflare - ekspozycja kamery

9.4.1 Instalacja cloudflared

9.4.2 Skrypt publikowania URL kamery

9.4.3 Cykliczne odnawianie URL

9.5 Konfiguracja crontab

9.5.1 Uzasadnienie parametrów

9.6 Problem z dostępnością sieci przy starcie

9.7 Podsumowanie konfiguracji Raspberry Pi

10 Wdrożenie i infrastruktura

10.1 Infrastruktura chmurowa

10.2 Inicjalizacja infrastruktury

10.3 Konteneryzacja - Dockerfiles

10.4 Docker Compose

10.5 Zmienne środowiskowe

10.6 Wdrożenie frontendu

10.7 Wdrożenie agenta

11 Testowanie systemu

11.1 Metodologia testowania

11.2 Testy jednostkowe - Auth Service

11.3 Testy integracyjne przez Postman

11.4 Test end-to-end - pełny przepływ druku

11.5 Testy wydajnościowe i obserwacje

12 Podsumowanie

12.1 Osiągnięcia

12.2 Napotkane trudności

12.3 Możliwości rozwoju

12.4 Wnioski

Literatura

- [1] Alexandre Joel Chorin. Numerical solution of the navier-stokes equations. *Mathematics of computation*, 22(104):745–762, 1968.
- [2] Dassault Systèmes. SOLIDWORKS – 3D CAD Design Software. <https://www.solidworks.com>. Dostęp: 2026.
- [3] Docker Inc. Docker Documentation. <https://docs.docker.com>. Dostęp: 2026.
- [4] Gina Häußge. Octoprint – The snappy web interface for your 3D printer. <https://octoprint.org>. Dostęp: 2026.
- [5] Maciej Matyka, Arzhang Khalili, and Zbigniew Koza. Tortuosity-porosity relation in porous media flow. *Physical Review E*, 78(2):026306, 2008.
- [6] Meta Platforms, Inc. React – The library for web and native user interfaces. <https://react.dev>. Dostęp: 2026.
- [7] Microsoft Corporation. Azure Blob Storage documentation. <https://learn.microsoft.com/en-us/azure/storage/blobs/>. Dostęp: 2026.
- [8] Microsoft Corporation. Azure Cosmos DB documentation. <https://learn.microsoft.com/en-us/azure/cosmos-db/>. Dostęp: 2026.
- [9] Microsoft Corporation. Azure IoT Hub documentation. <https://learn.microsoft.com/en-us/azure/iot-hub/>. Dostęp: 2026.
- [10] Microsoft Corporation. Azure Service Bus documentation. <https://learn.microsoft.com/en-us/azure/service-bus-messaging/>. Dostęp: 2026.
- [11] Microsoft Corporation. Typescript – JavaScript With Syntax For Types. <https://www.typescriptlang.org>. Dostęp: 2026.
- [12] OpenJS Foundation. Express – Fast, unopinionated, minimalist web framework for Node.js. <https://expressjs.com>. Dostęp: 2026.
- [13] OpenJS Foundation. Node.js – JavaScript runtime built on Chrome’s V8 engine. <https://nodejs.org>. Dostęp: 2026.

- [14] Poimandres. Zustand – Bear necessities for state management in React. <https://github.com/pmndrs/zustand>. Dostęp: 2026.
- [15] Python Software Foundation. Python 3 Documentation. <https://docs.python.org/3/>. Dostęp: 2026.
- [16] Tailwind Labs Inc. Tailwind CSS – Rapidly build modern websites without ever leaving your HTML. <https://tailwindcss.com>. Dostęp: 2026.
- [17] Evan You et al. Vite – Next Generation Frontend Tooling. <https://vitejs.dev>. Dostęp: 2026.

Załączniki

A Konfiguracja środowiska deweloperskiego

A.1 Wymagania

A.2 Pierwsze uruchomienie

B Struktura repozytoriów

C Endpointy API - podsumowanie